

```

import random

def displayMenu():
    """
    The functions displays the catalogue and asks user to perform an
    action
    All related functions are called based on the chosen action
    """

# create an empty list for orders to store the order data
orders=[]
# display the options to be executed
while True:
    print("Welcome to the Order Delivery tracking\n" \
          "1: Place Order\n" \
          "2: View Orders\n" \
          "3: Dispatch Orders\n" \
          "4: Track Order\n" \
          "5: Update Order Status\n" \
          "6: Search order by Name\n" \
          "7: Group orders by routes\n" \
          "8: Remove orders\n" \
          "9: Save Orders in a file\n" \
          "10: Calculate Total orders\n" \
          "11: Load Orders from a file\n" \
          "12: Exit")

    display = input("What would you like to do: ")

    if display == '1':
        customerData = {}
        customerData["name"] = input("Enter customer name: ").strip()
        customerData["address"] = input("Enter customer address: ").strip()
        customerData["product"] = input("Enter product: ").strip()
        addOrder(orders, customerData)

    if display == '2':
        printOrderSummary(orders)

    elif display == '3':
        dispatchOrders(orders)
        print("Order has been dispatched")

    elif display == '4':
        tid = input("Enter tracking ID to track: ").strip()
        trackOrder(orders, tid)

    elif display == '5':
        tid = input("Enter the tracking ID to update: ").strip()
        newOrderStatus = input("Enter the new status: ").strip()
        updateOrderStatus(orders, tid, newOrderStatus)

```

```

elif display == '6':
    custName = input("Enter Customer Name to be searched")
    searchOrdersByCustomer(orders, custName)

elif display == '7':
    routeDict = groupOrdersByRoute(orders)
    print("Orders grouped by route")
    for route, IDlist in routeDict.items():
        print(f"{route}: {IDlist}")

elif display == '8':
    tid = input("Enter the tracking ID to remove: ").strip()
    removeOrder(orders, tid)

elif display == '9':
    filename = input("Enter filename to save orders in txt")
    saveOrdersToFile(orders, filename)
    print("Orders saved successfully")

elif display == '10':
    totalOrders = int(calculateTotalOrders(orders))
    print("Total orders for today: ", totalOrders)

elif display == '11':
    filename = input("Enter filename to load orders from (e.g.,
orders.txt): ").strip()
    orders = loadOrdersFromFile(filename) # replace existing
orders with loaded data
    print("Orders loaded successfully.")

elif display == '12':
    break

else:
    print("Invalid entry")

def addOrder(orders: list, customerData: dict):
    """
    This function takes inputs from the user and stores the order details
    in dictionary
    TrackID is assigned to the order here from the trackID function
    orders is a list
    customerData is a dictionary with keys name, address and product
    """
    while True:
        if customerData.get("name"):
            break # If a non-empty name exists; exit the loop.
        else:
            customerData["name"] = input("Customer name is required.
Please enter customer name: ").strip()

```

```

while True:
    if customerData.get("address"):
        break # If a non-empty address exists; exit the loop.
    else:
        customerData["address"] = input("Customer address is
required. Please enter customer address").strip()
    getName = customerData["name"]
    getAddress = customerData["address"]
    getProduct = customerData.get("product","")

    trackID = generateTrackingId(getName)

    orderDetail = {'custID': trackID, 'Name':getName, 'Address':
getAddress,
                  'Product': getProduct, 'status': "Pending", 'route':
None}
    orders.append(orderDetail)

    print(f"Order placed for ID: {trackID}")

def generateTrackingId(getName: str) -> str:
    """
    This function generates a trackID for each order using first 3
letters of the Name
    and a random 3 digit number
    """
    firstThree = getName[:3].upper().ljust(3, 'X') # Fill X if name is
shorter than 3 letters
    lastThree = str(random.randint(100, 999))
    return firstThree + lastThree

def validateTrackingId(trackID):
    """
    The validity of he trackID is checked if its a 6 digit alphanumeric
string
    """
    while True:
        if (len(trackID)==6 and trackID.isalnum()):
            return trackID
        else:
            print("Invalid trackID")
            trackID = input("Enter a valid trackID")

def assignRoute(getAddress: str) -> str:
    """
    Assigns a route by looking for strings like north, south, east, west
in the address
    If not found then prompts to choose the best possible route
    """
    adrLower = getAddress.lower()
    if "north" in adrLower:
        return "Route A"
    if "south" in adrLower:
        return "Route B"

```

```

    if "east" in adrLower:
        return "Route C"
    if "west" in adrLower:
        return "Route D"
    else:
        return "Choose best possible route"

def dispatchOrders(orders: list):
    """
    This fucnction updates the order entry with the route and the status
    """
    for orderDetail in orders:
        orderDetail["route"] = assignRoute(orderDetail["Address"])
        orderDetail["status"] = "Dispatched"

def trackOrder(orders: list, trackID: str):
    """
    """
    for orderDetail in orders:
        if orderDetail["custID"] == trackID:
            print("Order found:")
            print(f"Tracking ID: {orderDetail['custID']}")
            print(f>Name: {orderDetail['Name']}")
            print(f"Address: {orderDetail['Address']}")
            print(f"Product: {orderDetail['Product']}")
            print(f>Status: {orderDetail['status']}")
            print(f"Route:{orderDetail['route']}")
            return
    print("Order not found")

def updateOrderStatus(orders: list, trackID:str, newOrderStatus: str):
    """
    """
    for orderDetail in orders:
        if orderDetail["custID"] == trackID:
            orderDetail["status"] = newOrderStatus
            print(f"Order {trackID} is updated to {newOrderStatus}")
            return
    print("Order not found")

def printOrderSummary(orders: list):
    """
    Print the list of orders till now
    """
    if len(orders) == 0:
        print("No orders to display")
        return

    print("\nOrder Summary")
    print("{:<10} {:<15} {:<25} {:<12} {:<10} {:<20}".format
        ("custID", "Name", "Address", "Product", "status", "route"))
    print("-"*110)

```

```

    for orderDetail in orders:
        print("{:<10} {:<15} {:<25} {:<12} {:<15} {:<20}".format
            (orderDetail["custID"],
            orderDetail["Name"],
            orderDetail["Address"],
            orderDetail["Product"],
            orderDetail["status"],
            orderDetail.get("route", "Unassigned")
        ))

def searchOrdersByCustomer(orders: list, custName: str):
    """

    """
    search = False
    for orderDetails in orders:
        if custName.lower() in orderDetails["Name"].lower():
            print("Customer Found")
            print(f"TrackID: {orderDetails['custID']}, Name:
{orderDetails['Name']}, Product:{orderDetails['Product']}, Status:
{orderDetails['status']}")
            search = True
    if not search:
        print("No orders found")

def calculateTotalOrders(orders: list) -> int:
    """

    """
    return len(orders)

def groupOrdersByRoute(orders: list) -> dict:
    """

    """
    grouped = {}
    for orderDetails in orders:
        route = orderDetails.get("route")
        if route not in grouped:
            grouped[route]=[]
        grouped[route].append(orderDetails["custID"])
    return grouped

def saveOrdersToFile(orders: list, filename: str):
    """
    Save the file with the latest entries
    The "w" creates a new file and overwrites an existing one
    """
    with open(filename, "w") as file:
        for orderDetails in orders:
            line =
f"{orderDetails['custID']}|{orderDetails['Name']}|{orderDetails['Address']

```

```

]]|{orderDetails['Product']}|{orderDetails['status']}|{orderDetails.get('
route','')}\n"
        file.write(line)

def removeOrder(orders: list, trackID: str):
    """
    Removes an order from the orders list based on the provided tracking
    ID.
    """
    for i, orderDetails in enumerate(orders):
        if orderDetails["custID"] == trackID:
            del orders[i]
            print(f"Order {trackID} removed.")
            return
    print("Order not found.")

def loadOrdersFromFile(filename: str) -> list:
    """
    Loads orders from a text file and reconstructs the orders list.
    Expects each line to be formatted as:
    tracking_id|name|product|status|route.
    """
    orders = []
    try:
        with open(filename, "r") as file:
            for line in file:
                parts = line.strip().split("|")
                if len(parts) < 4:
                    continue # Skip invalid lines
                order = {
                    "custID": parts[0],
                    "Name": parts[1],
                    "Address": parts[2],
                    "Product": parts[3],
                    "status": parts[4],
                    "route": parts[5] if len(parts) > 4 else None
                }
                orders.append(order)
    except FileNotFoundError:
        print("File not found.")
    return orders

def main():
    displayMenu()

if __name__ == '__main__':
    main()

```